

第29回 実践：Web ゲームプログラミング 8（追いつき・デバッグ・調整）

- 前回：最終必須仕様を一通り作った
- 今回：足りない部分を追いつき、エラーや動作を直す
- 次回：ゲームを完成させる

実習時のフォルダ構成

Web_名前 の中に 29 フォルダを作る。第 28 回の sample フォルダと exercise フォルダを、29 の中にコピーして、続きから作る。

```
Web_名前
└─ 29
   └─ sample          ← 第 28 回の sample をコピーする（今回は書き足さない）
      ├── index.html
      ├── style.css
      └─ main.js
   └─ exercise        ← 第 28 回の exercise をコピーする
      ├── index.html
      ├── style.css
      └─ main.js
```

- sample：第 28 回で完成している。今回は、sample/main.js にコードを書き足さない。
- exercise：自分のシューティングゲーム（演習 1～2、発展）

今回は、新しい書き方は学ばない。「作る」のではなく、「確認する」「直す」「追いつく」回にする。

講義内容

1. 最終必須仕様と、学んだ回

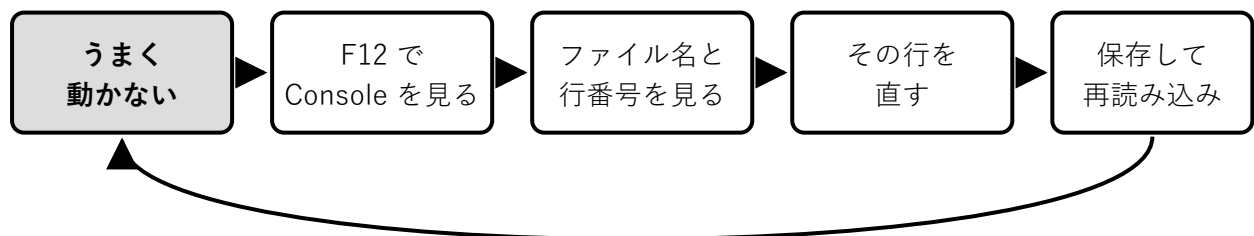
次の 10 項目が、最終回（第 30 回）までに自分のゲームで満たす「最終必須仕様」。どれも第 28 回までに学んでいる。

No.	仕様	学んだ回	<code>sample/main.js</code> ・ <code>index.html</code> で担当している部分
1	Canvas に自機を表示	第 21・22 回	<code>player</code> 、 <code>drawPlayer</code>
2	キーで自機を移動	第 23 回	キー入力、 <code>updatePlayer</code>
3	キーで弾を発射	第 24 回	<code>updateBullets</code> 、 <code>drawBullets</code>
4	敵が出現・移動	第 25 回	<code>updateEnemies</code> 、 <code>drawEnemies</code> 、 <code>removeEnemies</code>
5	弾と敵の当たり判定	第 26 回	<code>isHit</code> 、 <code>checkBulletHits</code>
6	敵に弾が当たると敵が消え、得点が増える	第 26 回	<code>checkBulletHits</code>
7	自機と敵の接触などで残機が減る	第 27 回	<code>checkPlayerHit</code>
8	得点・残機を HTML 上に表示し、jQuery で更新	第 26・27 回	<code>index.html</code> の表示欄、 <code>\$("#score").text(score)</code> など
9	残機 0 で HTML のゲームオーバーパネルを表示	第 28 回	<code>index.html</code> のパネル、 <code>style.css</code> 、 <code>checkPlayerHit</code> の <code>.show()</code>
10	リスタートボタンからゲームを再開	第 28 回	<code>resetGame</code> 、 <code>.on("click", resetGame)</code>

- 自分のゲームで、できていない項目があれば、その回のレジメと `sample` を見て追加する（演習 1）。

2. うまく動かないときの調べ方

うまく動かないときは、次の図の順に調べる。



まだ動かないときは、もう一度 Console を見る

エラーが出ないのに動かないときは、Console に変数名（`score` など）を入力して値を確かめる

(1) コンソールのエラーを読む

エラーは、Console に赤字で表示される。右側のファイル名と行番号（例：`main.js:85`）を見て、その行を確かめる。

エラーの表示（一部）	主な原因
ReferenceError: ○○ is not defined	変数名・関数名の綴りの間違い。作っていない変数や関数を使っている
ReferenceError: \$ is not defined	jQuery を読み込んでいない。jQuery の <code><script></code> が <code>main.js</code> より後ろにある。インターネットにつながっていない
TypeError: Cannot read properties of undefined (reading 'y')	配列のない番号（ <code>bullets[5]</code> など）の <code>.y</code> を使おうとした。 <code>for</code> の条件が <code><=</code> になっている、 <code>break</code> を忘れた、など
TypeError: Cannot read properties of null (reading 'getContext')	<code>getElementById("game")</code> の名前と、 <code><canvas></code> の <code>id</code> が違う
SyntaxError: Unexpected token '}'	<code>{ }</code> や <code>()</code> の数が合っていない。 <code>,</code> や <code>;</code> の書き忘れ・書きすぎ

- エラーが1つ出ると、そこから先の処理が実行されないことがある。上に表示されたエラーから順に直す。

(2) エラーが出ないのに、思いどおりに動かないとき

- コンソールに変数名を入力する：`score`、`lives`、`gameState`、`bullets.length`、`enemies.length` などを入力して Enter キーを押すと、その時点の値が分かる。
- `console.log` を一時的に書く：たとえば、`checkBulletHits` の `score += 100;` の下に `console.log(score);` を一時的に書くと、当たるたびに得点が表示される。確認したら、その行は消す。`update` の最初などに書くと、1秒に60回表示されて読めなくなるので、`if` の中など、たまにしか実行されない場所に書く。
- 1行ずつ `//` で止めてみる：あやしい行の先頭に `//` を付けて、動きがどう変わるかを見る。
- キーを離しても自機が止まらないとき：キーを押したままブラウザの外をクリックすると、キーを離したこと（`keyup`）が伝わらず、自機が動き続けることがある。ゲームの画面をクリックしてから、同じキーをもう一度押して離すと止まる。

(3) HTML・CSS・JavaScript・jQuery のつながりを確かめる

表示に関係する名前は、3つのファイルで同じにする必要がある。

HTML (<code>index.html</code>)	CSS (<code>style.css</code>)	JavaScript (<code>main.js</code>)
<code></code>		<code>\$("#score")</code>
<code></code>		<code>\$("#lives")</code>
<code><div id="gameOverPanel"></code>	<code>#gameOverPanel { display: none; }</code>	<code>\$("#gameOverPanel").show().hide()</code>
<code></code>		<code>\$("#finalScore")</code>
<code><button id="restartButton"></code>		<code>\$("#restartButton").on("click", resetGame)</code>
<code><canvas id="game"></code>		<code>document.getElementById("game")</code>

- jQuery では、`id` の前に `#` を付ける。付け忘れると、エラーは出ないが、表示が変わらない。

3. resetGame の確認

第 28 回で作った `resetGame` に戻し忘れがないかを、実際にゲームオーバー → リスタートして確かめる。

確認すること	確かめ方
得点は 0 に戻ったか	画面の SCORE、コンソールの <code>score</code>
残機は最初の数に戻ったか	画面の LIVES、コンソールの <code>lives</code>
弾・敵は残っていないか	画面、コンソールの <code>bullets.length</code> ・ <code>enemies.length</code>
自機は最初の位置に戻ったか	画面
HTML の表示も戻ったか	SCORE ・ LIVES の数字（変数だけ戻して、表示を戻し忘れているか）
パネルは消えたか	画面、 <code>resetGame</code> の <code>\$("#gameOverPanel").hide();</code>
もう一度遊べるか	自機が動き、弾が出て、敵が出るか（ <code>gameState</code> を <code>"play"</code> に戻したか）

4. 遊びやすさと見た目の調整

- **遊びやすさ**：自機の速さ（`player.speed`）、連射の間隔（`shotTimer = 10;`）、敵の出現間隔（`frameCount % 60`）、敵の速さ（`speed: 2`）、残機の数など。値を少しずつ変えて、遊んで確かめる。残機の数を変えるときは、ファイルの上の方と `resetGame`、`index.html` の最初の表示の 3 か所を変える。
- **色**：`ctx.fillStyle` には、CSS と同じ書き方で色を指定する（`#rrggbb` など）。背景を暗い色にして、自機・弾・敵を明るい色にすると見やすい。自機と敵は、はっきり違う色にする。

演習

`exercise/main.js`（自分のシューティングゲーム）で行う。今回の最優先は、演習 1（追いつき）と演習 2（不具合を直す力）。必須仕様 1～10 がそろい、不具合が直るまでは、色や遊びやすさの調整はしない。

演習 1 最終必須仕様に追いつく

- 講義内容 1 の表で、自分のゲームが 1～10 のどこまでできているかを確認する。
- できていない項目があれば、その回のレジメと `sample` を見て追加する。
- 講義内容 3 の表で、`resetGame` の戻し忘れがないかを確認して、直す。

演習 2 間違いを見つける

次の (1)～(4) には、それぞれ 1 か所ずつ間違いがある。どこが間違いで、ゲームがどうなるか（エラーが出るか、どんな動きになるか）を考えて、ノートに書く。分からないときは、`sample` をコピーしたファイルで、同じ間違いを実際を書いて確かめてもよい。

(1)

```
// 間違いのあるコード (1)
if (event.key === "arrowLeft") {
  isLeftPressed = true;
}
```

(2)

```
// 間違いのあるコード (2)
for (let i = 0; i <= bullets.length; i++) {
  ctx.fillRect(bullets[i].x, bullets[i].y, bullets[i].w, bullets[i].h);
}
```

(3)

```
// 間違いのあるコード (3)
function resetGame() {
  gameState = "play";
  score = 0;
  lives = 3;
  frameCount = 0;
  player.x = 184;
  bullets = [];
  enemies = [];
  shotTimer = 0;
  $("#score").text(score);
  $("#gameOverPanel").hide();
}
```

(4)

```
// 間違いのあるコード (4)
score += 100;
$("#score").text(score);
```

発展（任意）

演習 1・2 が終わり、必須仕様 1～10 がすべて動いている人だけが取り組む。

- 講義内容 4 を参考に、遊びやすさ（速さ・間隔など）と色を、自分で決めて調整する。自機・弾・敵が見分けやすいか、難しすぎないかを確かめる（遊びやすさの調整は、第 30 回の演習 2 で全員が行う）。